ZHAW School of Engineering
Institute of Mechatronic Systems

Betaflight – Offline Controller Tuning

Data-Driven Tuning of Rate and Altitude Controllers for FPV Drones

Bachelor Thesis

Yuri Bianchi, Dario Jurietti and Janick Dort

Supervisor: Michael Peter (pmic)
Co-Supervisor: Prof. Dr. Ruprecht Altenburger (altb)

April 13, 2026

Declaration of Originality

We hereby declare that this thesis is our own work and has been composed solely by the undersigned. We have not used any sources or aids other than those indicated. All passages taken verbatim or in substance from published or unpublished works of others have been clearly marked as such. This thesis has not been submitted, in whole or in part, for any other degree or examination at this or any other institution.

Winterthur, April 13, 2026

Yuri Bianchi

Dario Jurietti

Janick Dort

Zusammenfassung

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Abstract

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Contents

List of Figures

List of Tables

Preface

This thesis was carried out during the spring semester 2026 at the Institute of Mechatronic Systems (IMS) at the ZHAW School of Engineering. It continues the work started in the preceding project thesis conducted in the fall semester 2025.

We would like to thank our supervisor, Michael Peter, for...

1 Introduction

First-Person-View (FPV) racing drones require precise and responsive control to achieve stable flight behaviour, suppress disturbances such as propeller wash, and maintain agility during rapid manoeuvres. The flight performance strongly depends on the tuning of the flight controller's feedback loops. In particular, the parameters of the angular rate controller determine how quickly and accurately the drone reacts to pilot inputs and external disturbances.

In practice, controller tuning is still often performed manually through iterative trial-and-error procedures. This requires extensive flight testing and considerable piloting experience, making the process time-consuming and difficult to reproduce. Although modern flight controller firmware such as Betaflight provides extensive configuration options and detailed flight data logging, systematic tuning remains a challenge.

One commonly used tool for offline tuning is the PID-Toolbox. In this approach, flight logs are recorded during normal flights and later analysed to derive suitable controller parameters. However, the system excitation depends entirely on the pilot's inputs and the natural flight behaviour of the drone. As a result, not all relevant frequency ranges are sufficiently excited, which can limit the quality of the measurements.

In the approach proposed in this thesis, a chirp excitation signal is injected directly into the control loop. This allows the system to be excited over a wide range of frequencies in a controlled and systematic way, resulting in higher-quality measurements for system identification and controller tuning.

A previous project introduced a proof-of-concept approach for offline tuning of angular rate controllers based on recorded flight data. In the preceding project thesis, this concept was restructured and modularised and now serves as the foundation for the present work.

The objective of this bachelor thesis is to further refine and extend this approach into an offline tuning framework for multiple control loops used in Betaflight-based flight controllers. In addition to the existing angular rate controller tuner, the framework will be extended with code modules for tuning the angle controller, the altitude hold controller, and the position controller. This extends the tuning framework beyond the inner rate control loop and enables the analysis and tuning of higher-level control layers.

The framework will be implemented primarily in MATLAB and Python and will provide tools for analysing flight logs, estimating system dynamics, and generating controller parameters without requiring repeated flight tests. The software will be developed as an open-source library with accompanying documentation for the FPV community.

By providing a structured, data-driven approach to controller tuning, this project aims to simplify the tuning process and improve flight performance.

1.1 Initial situation

1.2 Task and Objective

2 Theoretical Foundations

This chapter provides the theoretical background for the methods used in this thesis. It covers the fundamental principles of drone dynamics and control, the use of chirp signals for system identification, and the signal processing techniques applied to estimate the transfer functions of the control loops. The concepts introduced here form the basis for the practical implementation and analysis of the control loops in the subsequent chapters.

2.1 Drone Types and Sensors

Unmanned Aerial Vehicles (UAVs) can generally be classified into different categories, such as fixed-wing, rotary-wing, and hybrid systems. This work focuses on multirotor UAVs, a subgroup of rotary-wing platforms, which generate lift through multiple rotors.

Among multirotor UAVs, the most common configurations are quadcopters, hexacopters, and octocopters, which differ in the number of rotors (see Fig. 2.1). Quadcopters use four rotors and represent the simplest fully controllable multirotor configuration. Due to their comparatively low mechanical complexity, good maneuverability, and widespread use in research and practical applications, they are particularly relevant for modeling and control tasks.

Hexacopters and octocopters extend this concept by using six and eight rotors, respectively. These configurations offer increased thrust, improved fault tolerance, and higher payload capacity, but they also involve greater weight, higher energy consumption, and more complex control allocation.

Since this work is concerned with the analysis and modeling of quadcopters, the quadcopter configuration is used as the reference platform throughout the remainder of this chapter.

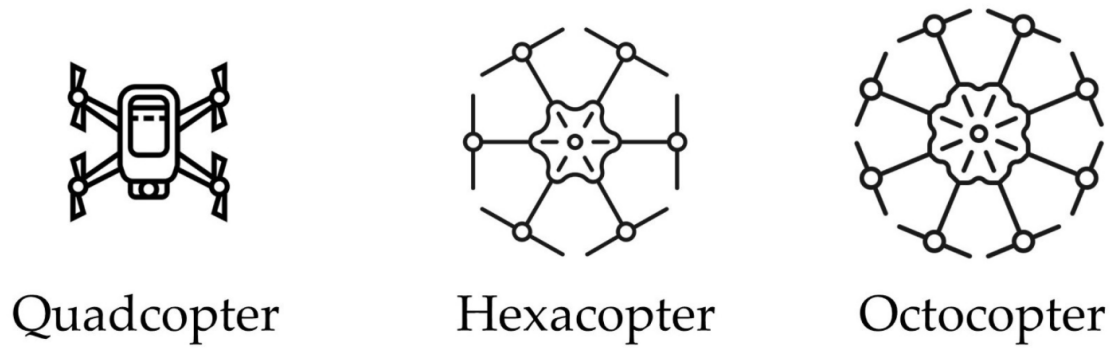

Figure 2.1: Rotary-wing multirotor UAV configurations: quadcopter (left), hexacopter (center), and octocopter (right).

2.1.1 Sensors

A quadcopter relies on a variety of sensors to measure its state and environment. The most common sensors include:

- **Inertial Measurement Unit (IMU):** The IMU consists of accelerometers and gyroscopes that measure linear acceleration and angular velocity. These measurements are used to estimate the drone's orientation and motion.
- **Barometer:** A barometer measures atmospheric pressure, which can be used to estimate altitude. This is important for altitude control.
- **GPS:** GPS provides position and velocity information and is mainly used for navigation and position control.
- **Magnetometer:** A magnetometer measures the Earth's magnetic field and is used to estimate the heading of the drone.
- **Electronic Speed Controllers (ESCs):** ESCs control the rotational speed of the motors and provide feedback for motor control.

These are the most important sensors used in a quadcopter. However, depending on the application, additional sensors such as cameras or distance sensors can also be used. In practice, the measurements of these sensors are often combined to obtain a more accurate estimate of the drone's state. These sensor data form the basis for stabilization, navigation, and control.

2.2 Drone Dynamics and Control Principles

A quadcopter is a dynamic system that uses four rotors to generate lift and control its motion. By adjusting the rotational speeds of the individual motors, the drone can control its orientation and movement in three-dimensional space.

Since no direct control surfaces are present, all forces and torques are generated by the propellers. This makes the system inherently unstable and requires continuous feedback control.

In this work, the focus lies on the rotational dynamics of the drone, which are described by the three degrees of freedom: roll, pitch, and yaw. These rotational motions are controlled by varying the thrust of the individual motors.

The following sections introduce the coordinate system, the propeller configuration, and the motor mixing approach, which together form the basis for the control strategy used in this work.

2.2.1 Coordinate System and Rotational Axes

The dynamics of a quadcopter are typically described using a body-fixed coordinate system, where the axes are defined as follows:

- The x-axis points forward, in the direction the drone is facing.
- The y-axis points to the right, perpendicular to the x-axis.
- The z-axis points downward, perpendicular to both the x- and y-axes.

This coordinate system follows the standard aerospace convention and moves together with the drone. All rotational motions are defined with respect to these axes.

The orientation of the quadcopter is described by three rotational degrees of freedom: roll, pitch, and yaw.

- **Roll** (ϕ) describes a rotation around the x-axis. A positive roll angle tilts the drone to the right.
- **Pitch** (θ) describes a rotation around the y-axis. A positive pitch angle tilts the drone forward.
- **Yaw** (ψ) describes a rotation around the z-axis. A positive yaw angle rotates the drone clockwise when viewed from above.

The sign conventions for these rotations follow the right-hand rule. This means that if the thumb of the right hand points in the direction of the axis, the curled fingers indicate the direction of positive rotation.

Figure 2.2 illustrates the body-fixed coordinate system and the associated rotational axes.

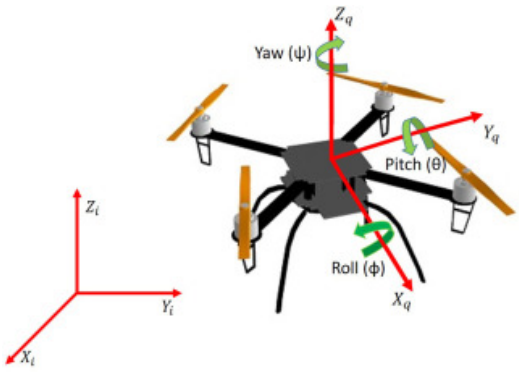

Figure 2.2: Body-fixed coordinate system and rotational axes of a quadcopter.

All this motion is controlled by adjusting the rotatioal speeds of the four motors, which generate the necessary fources and torques to achieve the desired orientation and movement. The specific configuration of the motors and the resulting control strategy are described in the following sections.

2.2.2 Force and Torque Generation

As already mentioned, a quadcopter generates all forces and torques through the thrust produced by the propellers. Therefore, the control of the drone is achieved by adjusting the speed of each motor, which in turn changes the thrust and the resulting forces and torques.

To roll and pitch the drone is relatively straightforward. This is achieved by increasing the speed of two motors on one side and decreasing the speed of the opposite motors. For example, to roll the drone to the right, the speed of the left motors is increased while the speed of the right motors is decreased. This creates a torque around the x-axis, causing the drone to tilt to the right. The same principle applies to pitch control, where the front and rear motors are adjusted to create a torque around the y-axis, as illustrated in Figure 2.3.

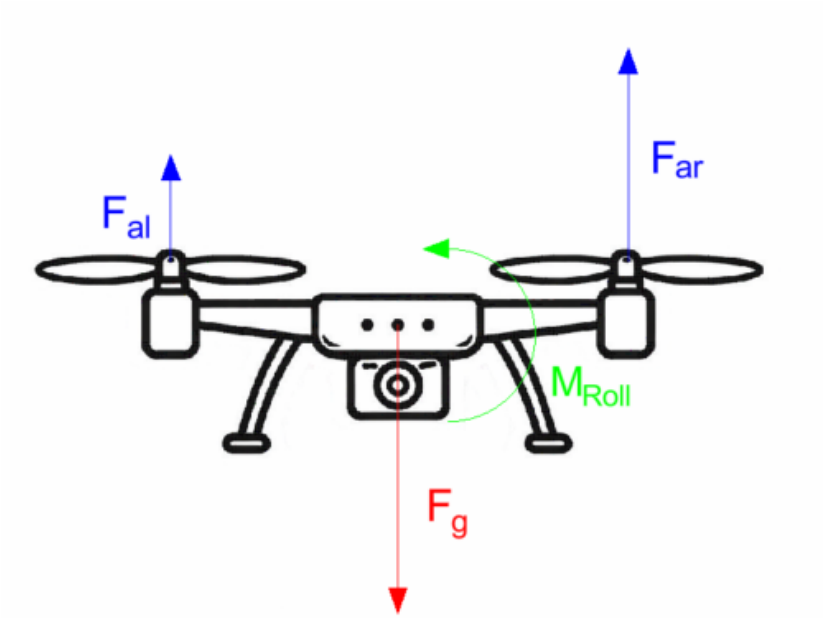

Figure 2.3: Illustration of roll and pitch control of a quadcopter through differential motor speeds, producing torques about the x- and y-axes.

To move along the x- and y-axes, the drone needs to tilt in the corresponding direction, which is achieved by adjusting the roll and pitch angles. For example, to move forward, the drone pitches forward by increasing the speed of the rear motors and decreasing the speed of the front motors. This creates a horizontal component of the thrust that propels the drone forward. Similarly, to move to the right, the drone rolls to the right by increasing the speed of the left motors and decreasing the speed of the right motors. Therefore, movement in the x- and y-directions is indirectly controlled through roll and pitch.

Yaw control is slightly more complex. Since the drone has no direct control surfaces, yaw is achieved by creating a torque around the z-axis. The propellers of quadcopters do not all rotate in the same direction but instead alternate between clockwise and counterclockwise rotation. As a result, the torque generated by the propellers also alternates in direction. To

create a yaw motion, the speed of the motors rotating in one direction is increased, while the speed of the motors rotating in the opposite direction is decreased. This results in a net torque around the z-axis, causing the drone to rotate in the desired direction, as shown in Figure 2.4.

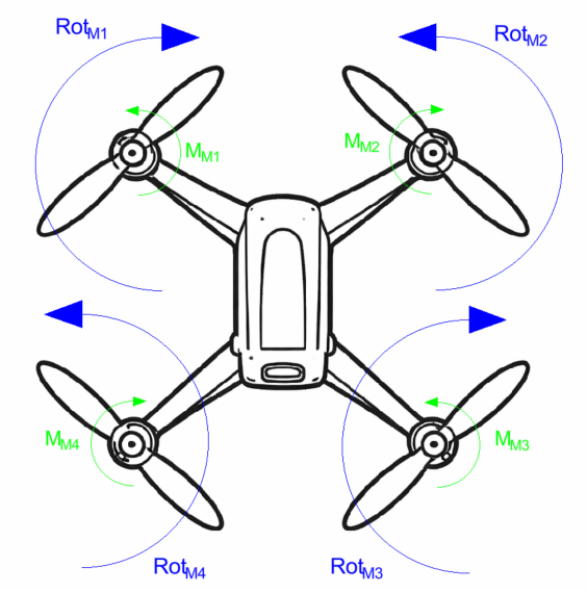

Figure 2.4: Top view of a quadcopter illustrating yaw control. Increasing the speed of one pair of counter-rotating propellers while decreasing the opposite pair generates a net torque about the z-axis.

If we combine all these control principles, it becomes clear that the drone can perform all maneuvers by adjusting the speeds of the four motors. For this purpose, the software mixes the desired roll, pitch, yaw, and throttle commands into individual motor speed commands. This process is known as motor mixing or control allocation and is a crucial part of the flight controller's functionality. The simplified motor mixing for a quadcopter can be represented by the following equations:

$$M_1 = T + \phi + \theta - \psi, \tag{2.1}$$
$$M_2 = T - \phi + \theta + \psi, \tag{2.2}$$
$$M_3 = T - \phi - \theta - \psi, \tag{2.3}$$
$$M_4 = T + \phi - \theta + \psi, \tag{2.4}$$

where M_i represents the speed command for the i-th motor, T is the throttle command, and ϕ, θ, and ψ are the desired roll, pitch, and yaw commands, respectively. The specific coefficients in these equations depend on the configuration of the drone and the direction of rotation of the motors.

2.2.3 Flight Control

With the principles of force and torque generation in mind, the flight control system can now be introduced. Up to this point, the fundamental concepts required to understand the behavior of a quadcopter have been described. The following section marks the transition to the main focus of this thesis.

The general structure of the flight control system is illustrated in Figure 2.5. It shows how the desired maneuver (setpoint) is processed by the controller, which computes motor commands to influence the drone. The resulting motion is measured by sensors and fed back into the system, forming a closed control loop.

The flight control system consists of multiple control loops that regulate the drone's behavior. These control loops are the primary subject of this work and will be analyzed and tuned in the following sections.

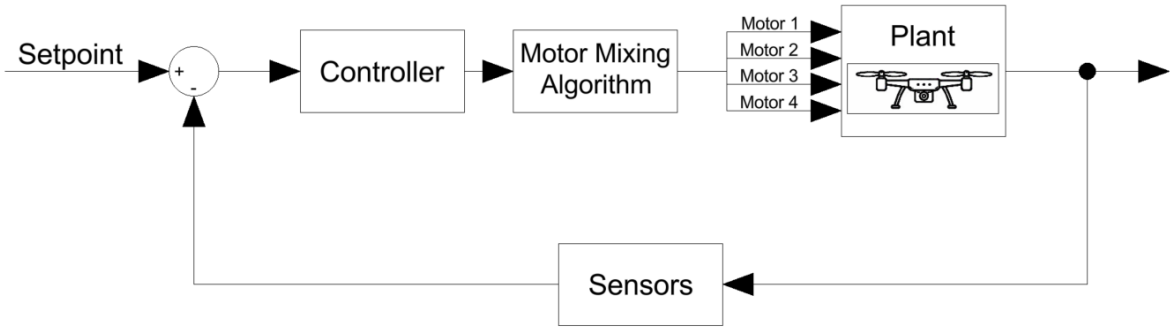

Figure 2.5: General structure of the quadcopter flight control system showing the closed control loop between setpoint, controller, motors, drone dynamics, and sensor feedback.

2.3 System Identification Using Chirp Signals

In this chapter, the theoretical background of the approach used for the estimation and calculation of the different controll loops is described. Since the fundamental approuch of all the control loops is the same, the general approach is described here and the specific details of each control loop are described in the Methodology chapter.

To support the theoretical concepts, a simplified simulation model is added. The simulation is gradually developed throughout this chapter, allowing the reader to follow the modelling process step by step and to better understand the behaviour of the control system. This approach should provide a clear and practical understanding of the control loops and the signal processing techniques used in the estimation and calculation of the control loops.

The detailed application of these principles to the different control loops implemented in Betaflight is discussed in the following chapter.

2.3.1 Chirp Signal

A chirp signal is a signal in which the frequency increases or decreases over time. It is commonly used in system identification and control engineering to analyze the response of a system over a range of frequencies.

The main advantage of a chirp signal is that it allows a wide frequency range to be excited within a single experiment. Instead of testing individual frequencies one by one, the system is continuously stimulated with increasing frequency. This makes the approach both efficient and well suited for practical applications.

By observing how the system responds to this input, it becomes possible to analyse its frequency-dependent behaviour. In particular, the measured input and output signals can be used to estimate the frequency response and, consequently, the transfer function of the system. This provides valuable insight into the system dynamics and forms the basis for the methods described in the following sections.

2.3.1.1 Mathematical Definition

The instantaneous value of the chirp signal is defined as

$$x(t) = A \cdot \sin(\varphi(t)), \tag{2.5}$$

where A is the amplitude. The amplitude determines the excitation strength of the signal and therefore influences the signal-to-noise ratio of the measurement.

The phase is defined as

$$\varphi(t) = 2\pi \int_0^t f(\tau)\, d\tau, \tag{2.6}$$

and represents the accumulated instantaneous frequency over time, where $f(\tau)$ denotes the instantaneous frequency at time τ. The specific evolution of the frequency over time will be introduced in the following sections.

In practical implementations such as **Betaflight**, the phase is wrapped once it reaches 2π. This keeps the numerical values bounded and results in a sawtooth-shaped wrapped phase signal, which serves as the argument of the sinusoidal excitation.

The behaviour of the chirp signal is determined by several parameters defining the frequency sweep.

For the simulation model introduced in this chapter, the following parameter values are used:

- f_0 — starting frequency of the chirp signal (simulation: 1 Hz)
- f_1 — final frequency of the chirp signal (simulation: 600 Hz)
- T — total duration of the frequency sweep (simulation: 20 s)
- f_s — sampling frequency used in the discrete-time simulation (simulation: 2000 Hz)
- A — amplitude of the chirp signal (simulation: 230)

These parameters are chosen to reflect the typical operating range of the angular rate and the attitude while ensuring that the relevant frequency dynamics are sufficiently excited.

These parameters directly influence the quality of the system excitation and therefore the accuracy of the subsequent frequency response estimation.

2.3.1.2 Common Frequency Profiles

The chirp signal can be generated using different frequency profiles, which define how the frequency changes over time. The two most common profiles are the linear chirp and the exponential chirp.

Linear chirp:

For a linear chirp, the instantaneous frequency increases linearly over time.

The instantaneous frequency is therefore given by

$$f(t) = f_0 + \frac{f_1 - f_0}{T} \cdot t, \tag{2.7}$$

and the corresponding phase follows by integrating the frequency over time.

$$\varphi(t) = 2\pi \left(f_0 t + \tfrac{1}{2} k t^2 \right). \tag{2.8}$$

Figure 2.6 illustrates the resulting chirp signal, the instantaneous frequency $f(t)$, and the corresponding phase trajectory $\varphi(t)$ for the simulation parameters introduced above.

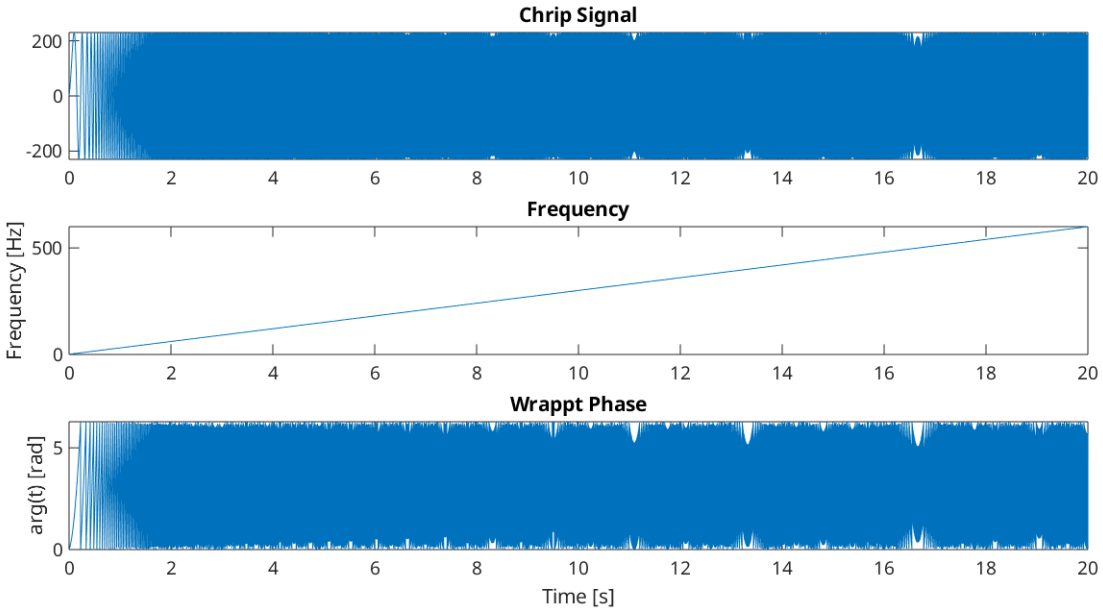

Figure 2.6: Linear chirp signal used for the simulation.

The apparent gaps visible in the chirp signal and in the wrapped phase are not actual discontinuities. They result from the finite sampling time of the simulation and the limited resolution of the plotted data.

Exponential chirp (as used in Betaflight):

For an exponential chirp, the instantaneous frequency increases exponentially from the starting frequency f_0 to the final frequency f_1 over the time interval T. This means that lower frequencies are covered more slowly, while higher frequencies are reached more rapidly.

The instantaneous frequency is defined as

$$f(t) = f_0 \left(\frac{f_1}{f_0}\right)^{t/T}. \tag{2.9}$$

Compared to the linear chirp, this results in a more uniform excitation in logarithmic frequency space, which is particularly useful when analysing systems using Bode plots.

The corresponding phase trajectory is obtained by integrating the frequency

$$\varphi(t) = \frac{2\pi T f_0}{\ln(f_1/f_0)} \left[\left(\frac{f_1}{f_0}\right)^{t/T} - 1\right]. \tag{2.10}$$

These expressions define the phase trajectory $\varphi(t)$ used both in the simulations and in the time-dependent frequency shifting approach.

The resulting exponential chirp signal, the instantaneous frequency, and the phase trajectory are illustrated in Figure 2.7.

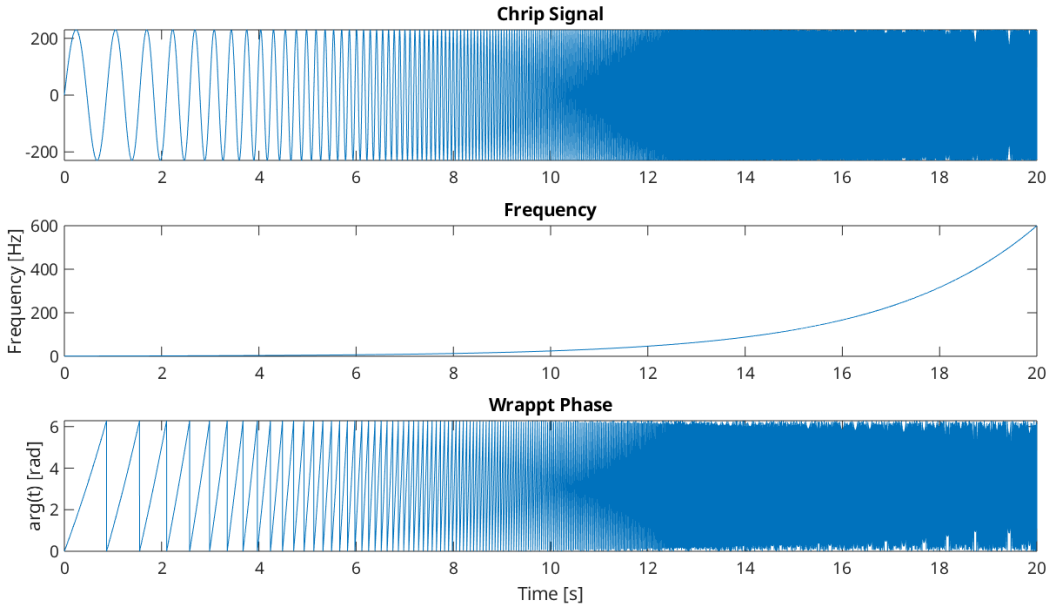

Figure 2.7: Exponential chirp signal used for the simulation.

As with the linear chirp, the apparent gaps in the signal and the wrapped phase are caused by the discrete sampling time and the limited resolution of the plot.

2.3.1.3 Lag Filter for Chirp Input Shaping

Before the chirp signal is injected into the control loop, it is conditioned by a filter to address the differentiating behavior of the controller effort at low frequencies. This differentiating characteristic causes rising gain at low frequencies, which would lead to excessive motor response at the start of the chirp sweep and potentially cause saturation, compromising the quality of system identification data. The filter reduces low-to-mid frequency components of the chirp signal before injection, such that the combined response produces approximately uniform excitation across the frequency range.

The filter is configured with a pole at 3 Hz and a zero at 30 Hz. This configuration corresponds to the transfer function

$$H(s) = \frac{1 + s/\omega_z}{1 + s/\omega_p} \tag{2.11}$$

where $\omega_z = 2\pi \cdot 30$ rad/s and $\omega_p = 2\pi \cdot 3$ rad/s represent the zero and pole frequencies respectively. The pole introduces the lag component by creating magnitude reduction at frequencies above its cutoff frequency. The zero limits this reduction by introducing magnitude amplification at frequencies above its cutoff frequency. However, since the pole frequency is lower than the zero frequency, the pole's contribution dominates throughout the primary control bandwidth, resulting in a net lag filter. It is important to note that in the Betaflight implementation, the pole frequency (ω_p) must not be set higher than the zero frequency (ω_z), as this would create a lead filter that amplifies high frequencies and could potentially damage the motors.

For the simulation shown in Figure 2.8, the same parameter values introduced in the previous section were used ($f_0 = 1\,\text{Hz}$, $f_1 = 600\,\text{Hz}$, $T = 20\,\text{s}$, $f_s = 2000\,\text{Hz}$). The excitation signal is generated using the exponential chirp formulation described earlier.

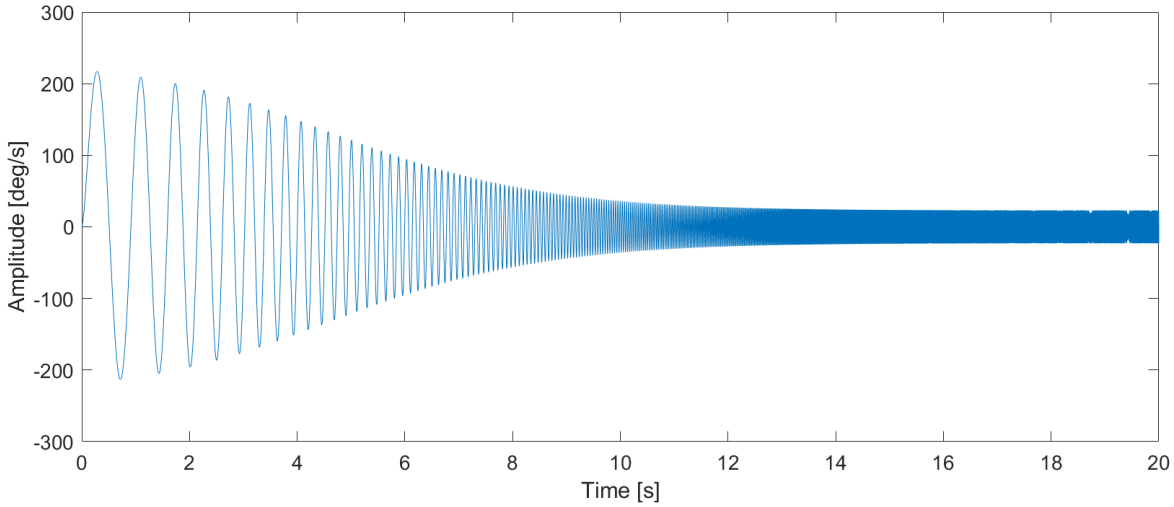

Figure 2.8: Exponential chirp signal after conditioning with the lag filter used for chirp input shaping.

2.3.2 Signal Processing

To illustrate the influence of the applied signal-processing steps, a simulation is used as a reference. The simulated system represents a simplified model of the plant of the drone's angular-rate controller. It consists of an integrator, a low-pass filter, and a dead-time component. These elements capture the main dynamics of the system and are commonly used in control engineering.

The overall transfer function of the system is given by

$$G(s) = \frac{\omega_1}{s} \cdot \frac{1}{1 + s/\omega_2} \cdot e^{-s/\omega_t}. \tag{2.12}$$

The integrator describes how differences in rotor thrust affect the angular rate over time. The first-order term represents the low-pass behavior introduced by the electronics and actuator dynamics. The dead-time component accounts for delays in the control loop.

For the simulation, the parameters are chosen as

$$\omega_1 = 16 \cdot 2\pi, \qquad \omega_2 = 13 \cdot 2\pi, \qquad \omega_t = 60 \cdot 2\pi. \tag{2.13}$$

The structure of the simulation model is illustrated in Figure 2.9.

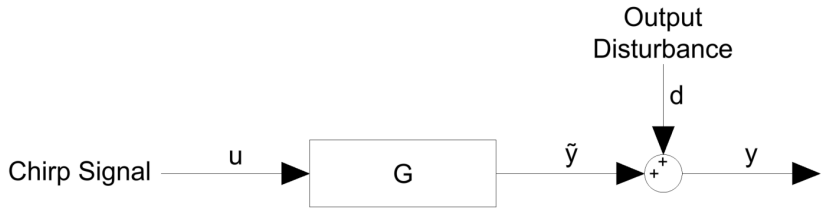

Figure 2.9: Simulation model used for the signal processing analysis.

The chirp signal $u(t)$ is applied to the system, resulting in the ideal output $\tilde{y}(t)$.

To emulate realistic measurement conditions, additive disturbance noise $d(t)$ is superimposed on the output, such that

$$y(t) = \tilde{y}(t) + d(t). \tag{2.14}$$

In the simulation, the disturbance is modelled as additive white Gaussian noise with a standard deviation of $\sigma_d = 2\,\mathrm{deg/s}$. Based on these simulated signals, the following signal-processing steps are applied and evaluated.

2.3.2.1 Baseband Filtering via Frequency Rotation

Measured signals are often affected by noise and unwanted high-frequency components, which can obscure relevant information. This is also the case in the simulated signals considered here, where additive noise is present. To reduce these disturbances while preserving the phase characteristics, a frequency-shifting filtering approach is used.

The method shifts the signal to baseband, applies a low-pass filter, and then shifts it back to its original frequency range, enabling focused filtering around the frequency of interest.

The underlying principle can be understood using the Fourier shift theorem. Multiplication of a signal $f(t)$ with a complex exponential results in a shift of its spectrum

$$e^{i2\pi\xi_0 t} f(t) \iff \hat{f}(\xi - \xi_0),$$

which shifts the spectrum by the frequency ξ_0.

In this case, the signal of interest is a chirp signal with a continuously changing instantaneous frequency. Therefore, the frequency shift has to be time-dependent. This is achieved using a complex phasor

$$p(t) = e^{i\phi(t)},$$

where $\phi(t)$ describes the instantaneous phase of the chirp.

Multiplying the measured signal $x(t)$ with this phasor or its complex conjugate results in a rotation of the signal spectrum toward baseband

$$y_R(t) = x(t)p(t), \qquad y_Q(t) = x(t)p^*(t).$$

After this rotation, the dominant signal component is centered around 0 Hz, enabling efficient filtering using a low-pass filter.

Once the signal has been shifted to baseband, a low-pass filter is applied to remove high-frequency noise components. Since the useful signal is concentrated near zero frequency, the filter can effectively suppress unwanted spectral components outside this region.

To avoid phase distortion, forward-backward filtering is employed. This approach applies the filter in both forward and reverse directions, resulting in zero-phase filtering.

The filtering operation yields the filtered baseband signals

$$y_R(t) \to \tilde{y}_R(t), \qquad y_Q(t) \to \tilde{y}_Q(t),$$

which contain the desired signal components with reduced noise.

After filtering, the signal is shifted back to its original frequency range. This is achieved by multiplying the filtered signals with the corresponding inverse phasors and combining the results to obtain a real-valued signal

$$x_f(t) = \mathrm{Re}\left(\frac{1}{2}\left(\tilde{y}_R(t)p^*(t) + \tilde{y}_Q(t)p(t)\right)\right).$$

This step reverses the initial frequency rotation and reconstructs a filtered version of the original signal.

The filtered signal is then used for further analysis. By reducing the noise before the spectral estimation, the signal-to-noise ratio is improved, leading to more reliable results.

The effect of the filtering approach is illustrated in Fig. 2.10. The upper plot shows the unfiltered signal, where noise becomes dominant at later time instances corresponding to higher chirp frequencies, while the lower plot shows the filtered signal.

The method is based on shifting the signal to baseband, applying a low-pass filter, and shifting it back to its original frequency range. A second-order Butterworth low-pass filter with a cutoff frequency of 10 Hz is used. It reduces noise especially at higher frequencies while preserving the main signal characteristics.

The filtered signal closely follows the underlying system response. A slight amplitude reduction at higher frequencies can be observed, which is consistent with the low-pass behavior.

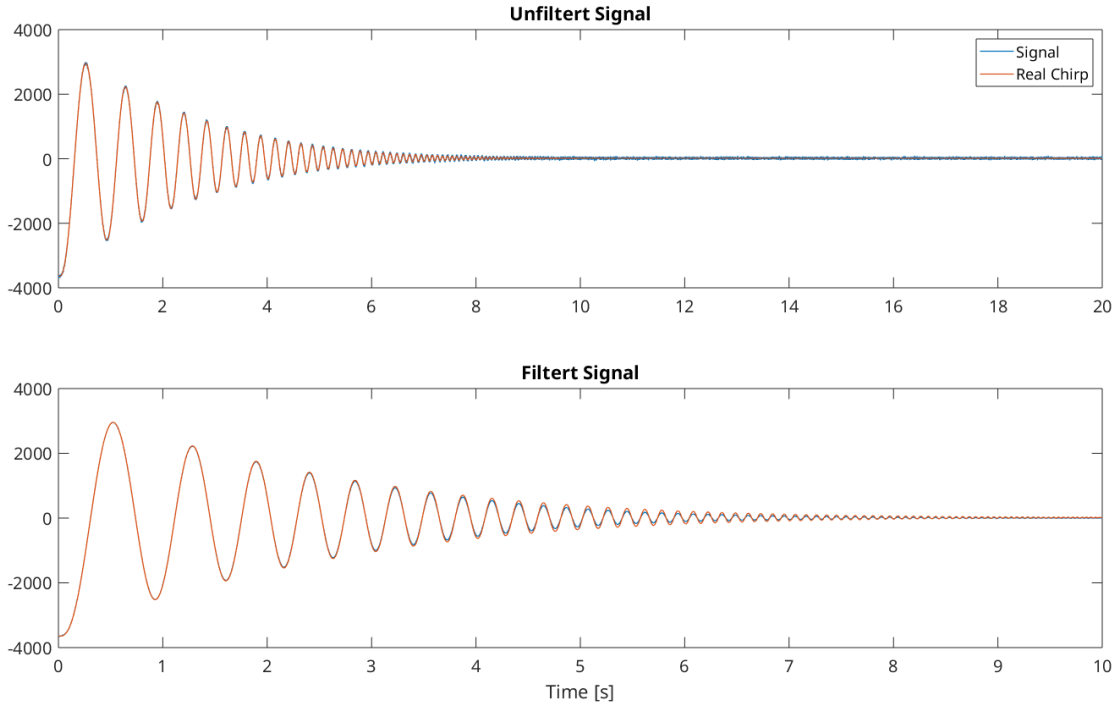

Figure 2.10: Time-domain signal before (top) and after (bottom) baseband filtering.

2.3.2.2 Frequency Response Estimation

To estimate the transfer function, the relation between input and output signals is analyzed in the frequency domain. For a linear time-invariant (LTI) system, this relation can be written as

$$G(\omega) = \frac{Y(\omega)}{U(\omega)},$$

where $U(\omega)$ and $Y(\omega)$ denote the Fourier transforms of the input signal $u(t)$ and the output signal $y(t)$, respectively.

In practice, however, the measured output signal is affected by disturbances. Even after applying the baseband filtering, residual noise remains, especially at higher frequencies. The measured output can therefore be written as

$$Y(\omega) = G(\omega)U(\omega) + D_y(\omega),$$

where $D_y(\omega)$ represents an additive disturbance. A direct estimation by dividing $Y(\omega)$ by $U(\omega)$ leads to

$$\frac{Y(\omega)}{U(\omega)} = G(\omega) + \frac{D_y(\omega)}{U(\omega)},$$

which shows that the estimate is directly influenced by noise.

To reduce the sensitivity to noise, spectral quantities can be introduced. However, computing these directly from the Fourier transforms leads to the same issues as the direct division $Y(\omega)/U(\omega)$.

A more robust approach is to estimate the spectral densities using averaging techniques, which reduce the influence of disturbances. This is typically achieved using methods such as Welch's method, which is described in the following section.

In particular, the estimation becomes unreliable in frequency regions where the input signal is small, since the disturbance term can dominate the ratio.

2.3.2.3 Spectral Estimation Using Welch's Method

To obtain reliable estimates of the spectral densities, Welch's method is applied. As discussed in the previous section, direct computation from finite data is sensitive to noise. Therefore, averaging techniques are required to approximate the expected values of the spectral quantities.

The spectral densities describe how the signal energy is distributed over frequency and form the basis for estimating the transfer function.

Let $u[n]$ and $y[n]$ denote the discrete-time input and output signals of length N. In Welch's method, the signals are divided into K overlapping segments of length L. Each segment is multiplied by a window function $w[n]$ to reduce spectral leakage.

The k-th segment of the input signal is given by

$$u_k[n] = u[n + kD]\, w[n], \qquad n = 0, \ldots, L-1, \tag{2.15}$$

where D defines the shift between segments and determines the overlap.

For each segment, the discrete Fourier transform (DFT) is computed as

$$U_k(\omega) = \sum_{n=0}^{L-1} u_k[n] e^{-j\omega n}, \qquad Y_k(\omega) = \sum_{n=0}^{L-1} y_k[n] e^{-j\omega n}. \tag{2.16}$$

To obtain a stable representation in the frequency domain, the squared magnitude of the Fourier transform is used. This corresponds to the signal power and makes the spectral estimate less sensitive to noise.

The periodogram estimates are defined as

$$P_{uu,k}(\omega) = \frac{1}{L}|U_k(\omega)|^2, \qquad P_{yu,k}(\omega) = \frac{1}{L} Y_k(\omega) U_k^*(\omega) \tag{2.17}$$

Welch's method reduces the variance of the estimates by averaging over all segments. The resulting spectral densities are given by

Welch's method reduces the variance of the estimates by averaging over all segments. The resulting spectral densities are given by

$$S_{uu}(\omega) = \frac{1}{K} \sum_{k=1}^{K} P_{uu,k}(\omega), \qquad S_{yu}(\omega) = \frac{1}{K} \sum_{k=1}^{K} P_{yu,k}(\omega), \tag{2.18}$$

Using these quantities, the transfer function estimate is computed as

$$G(\omega) = \frac{S_{yu}(\omega)}{S_{uu}(\omega)}. \tag{2.19}$$

In our example, the spectral densities are estimated using Welch's method with a segment length of $L = 4096$ samples and an overlap of 90%. A Hanning window is applied to each segment to minimize spectral leakage.

The estimated transfer function is shown in Fig. 2.11. Compared to the direct estimation, a significantly better agreement with the true system response can be observed over a wider frequency range.

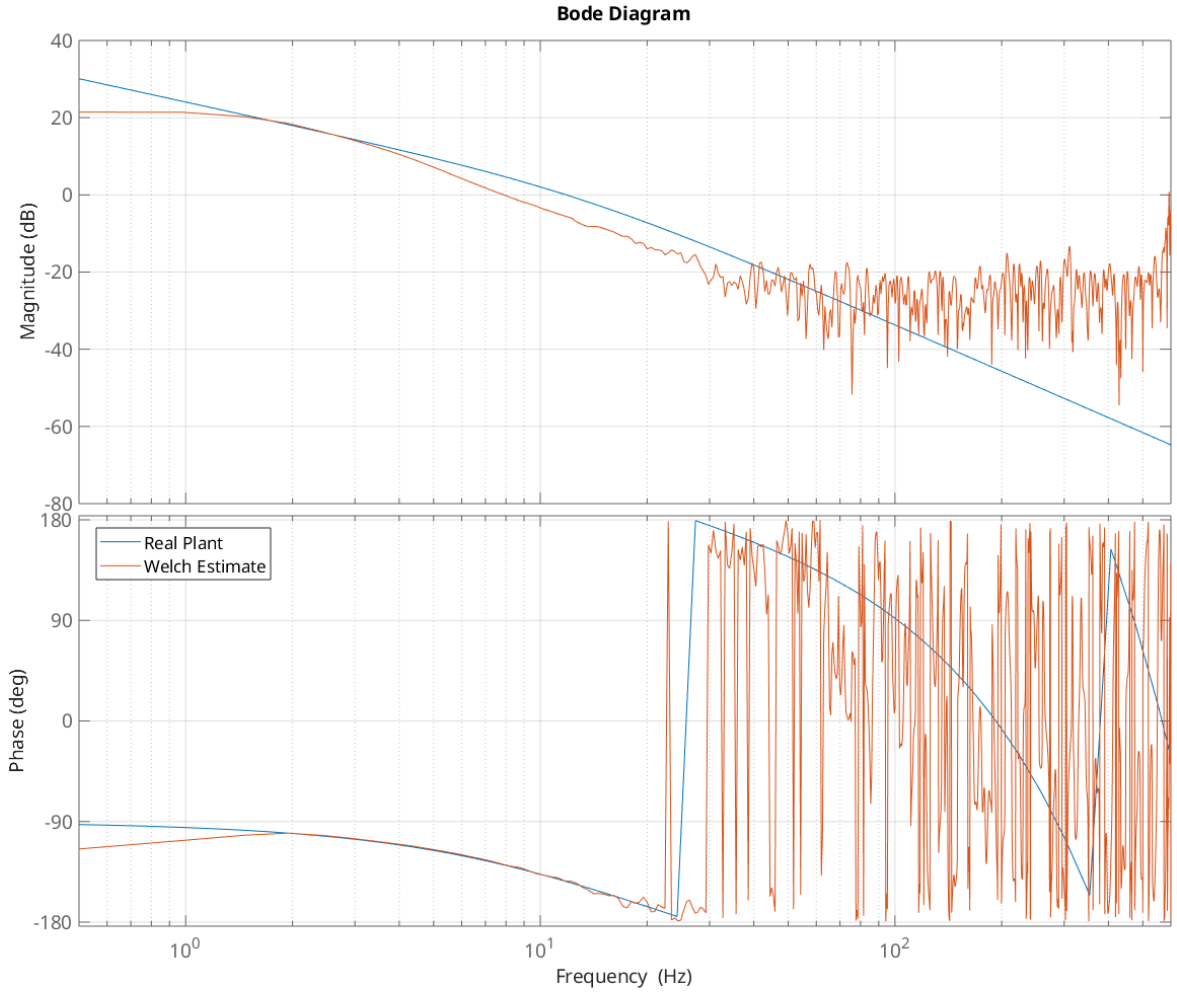

Figure 2.11: Bode plot of the true and estimated transfer function using Welch's method. The agreement is improved due to the reduced influence of noise.

To further assess the quality of the estimate, the coherence function is evaluated. The coherence is defined as

$$\gamma^2(\omega) = \frac{|S_{yu}(\omega)|^2}{S_{uu}(\omega)\, S_{yy}(\omega)},$$

and takes values in the range $0 \leq \gamma^2(\omega) \leq 1$. It provides a frequency-dependent measure of how strongly the output is linearly related to the input.

The coherence is shown in Fig. 2.12. Values close to one indicate a reliable estimate, whereas lower values indicate that noise or disturbances dominate the measurement.

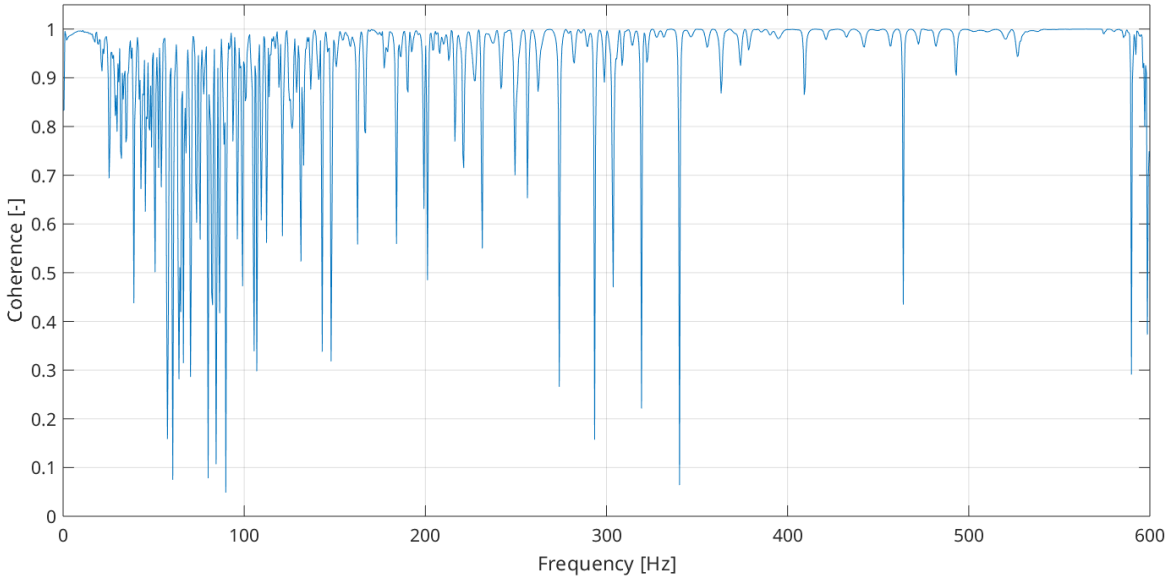

Figure 2.12: Magnitude-squared coherence between input and output signals. High values indicate a reliable transfer function estimate.

2.4 Control Loop Calculations

Although the flight controller consists of several nested control loops, their fundamental structure is identical. Each loop follows the same basic feedback control principle and differs mainly in the controlled variable and the chosen controller parameters. Therefore, instead of analysing every loop individually from the beginning, a simplified model is used to explain the underlying calculations and controller behaviour.

The simplified control structure used for the analysis consists of a controller C and a plant P, which describes the dynamic behaviour of the system being controlled. The reference input r is compared with the system output y to form the control error e. Based on this error, the controller generates the control signal u, which drives the plant. In addition, the model considers both input and output disturbances, allowing a more realistic representation of the system behaviour, as illustrated in Figure 2.13.

Using this simplified model allows the general behaviour of the control loop and the calculation of the controller terms to be explained in a clear and structured way. The same principles are later applied to the individual control loops implemented in the flight controller.

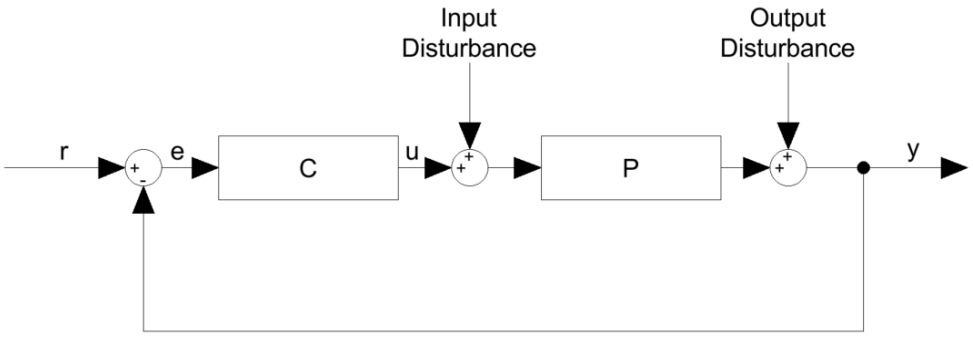

Figure 2.13: Simplified feedback control loop with controller C and plant P, including reference r, error e, control signal u, and input/output disturbances.

2.4.1 Calculation of the Control Loop Components

After the signal processing steps, the control loop components can be calculated. As in the signal processing stage, the goal is to mitigate the influence of disturbances and noise on the estimated control loop components. Therefore, only the input signal of the control loop is used for the estimation.

The advantage of this approach is twofold:

- The input signal is not affected by the back-calculation within the control loop and is therefore not influenced by disturbances or noise present inside the loop.

- The input signal is not subject to filtering effects, since it is not measured but taken directly from the software. Consequently, it is free from measurement noise and external disturbances.

This approach enables a more precise estimation of the transfer function. However, it also implies that the transfer functions of the individual control loop components cannot be obtained directly. Instead, they must be derived through mathematical manipulations.

Consider a simplified control loop consisting of a controller $C(s)$ and a plant $P(s)$, as illustrated in Figure 2.13. The closed-loop transfer function from the reference input $R(\omega)$ to the output $Y(\omega)$ is given by

$$G_{yr}(\omega) = \frac{Y(\omega)}{R(\omega)} = \frac{C(\omega)P(\omega)}{1 + C(\omega)P(\omega)}. \tag{2.20}$$

The transfer function from the reference input to the controller output $U(\omega)$ is

$$G_{ur}(\omega) = \frac{U(\omega)}{R(\omega)} = \frac{C(\omega)}{1 + C(\omega)P(\omega)}. \tag{2.21}$$

By taking the ratio of these two transfer functions, the plant transfer function can be determined:

$$P(\omega) = \frac{G_{yr}(\omega)}{G_{ur}(\omega)} = \frac{\dfrac{C(\omega)P(\omega)}{1 + C(\omega)P(\omega)}}{\dfrac{C(\omega)}{1 + C(\omega)P(\omega)}} = P(\omega). \tag{2.22}$$

To determine the controller, the same principle is applied. The controller transfer function can be calculated as

$$C(\omega) = \frac{\dfrac{C(\omega)}{1 + C(\omega)P(\omega)}}{1 - \dfrac{C(\omega)P(\omega)}{1 + C(\omega)P(\omega)}} \tag{2.23}$$

These relationships enable the identification of both the plant and the controller from measurable closed-loop signals. They provide the theoretical foundation for the analysis of the individual control loops.

2.4.2 Closed-Loop Performance Metrics

After determining the plant transfer function, the next step is to analyze the behaviour of the overall system when a controller is applied. This is achieved by evaluating the closed-loop transfer functions, which describe the system response, disturbance sensitivity, disturbance rejection capability, and the required controller effort. These measures provide valuable insights into the stability, robustness, and performance of the controlled system.

The closed-loop transfer function from the reference input to the output is given by

$$G_{yr}(\omega) = \frac{C(\omega)P(\omega)}{1 + C(\omega)P(\omega)}.$$

The sensitivity function, which describes how sensitive the system output is to disturbances and model uncertainties, is defined as

$$S(\omega) = \frac{1}{1 + C(\omega)P(\omega)}.$$

The compliance function, representing the system response to disturbances acting on the plant, is given by

$$SP(\omega) = P(\omega)\, S(\omega).$$

Finally, the controller effort describes the influence of the reference input on the controller output and is defined as

$$SC(\omega) = C(\omega)\, S(\omega).$$

2.4.3 Compensation of the I-Term

The integral term of a PID controller is used to compensate for steady-state errors by integrating the control error over time. In flight control systems, this helps to counteract constant disturbances such as motor imbalances or aerodynamic effects. During fast stick movements or aggressive maneuvers, however, the integral term can accumulate too much error.

In these situations, the accumulated I-term no longer represents a disturbance but instead opposes the intended motion. When the maneuver ends, this can lead to overshoot, bounce-back effects, or slow settling of the system.

To reduce these effects, Betaflight uses a feature called I-Term Relax. This mechanism temporarily limits the integration of the I-term when rapid changes are detected. Depending on the configuration, these changes are identified either from the control setpoint or from the

measured gyroscope signal. When the detected change exceeds a defined threshold, the I-term integration is reduced or paused.

This allows the controller to respond quickly to fast pilot inputs while preventing excessive buildup of the integral term. During slower movements and steady operation, normal integration is restored. As a result, I-Term Relax improves flight behavior by reducing overshoot and bounce-back while maintaining good disturbance rejection.

For analysis and controller tuning, the effect of I-Term Relax must be considered, as it modifies the controller behavior depending on the current flight conditions.

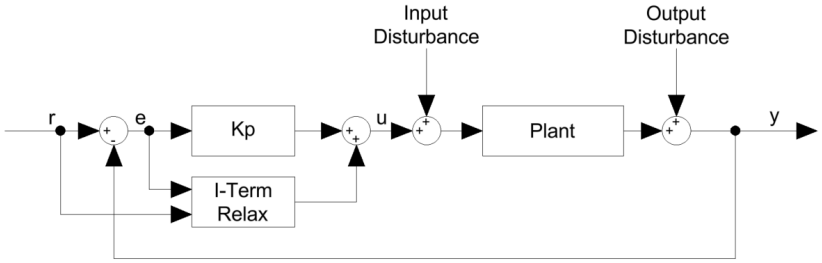

Figure 2.14: Simplified closed loop system with I-term Relax

The measured controller transfer function is compared with an analytically derived controller transfer function. The effect of the I-Term Relax algorithm is therefore included implicitly in the measurement-based result.

To this end, a compensation factor is defined as the ratio between the measured PI controller transfer function and its analytical counterpart:

$$C_{\mathrm{PI,com}}(\omega) = \frac{C_{\mathrm{PI}}(\omega)}{C_{\mathrm{PI,ana}}(\omega)} \tag{2.24}$$

This frequency-dependent compensation factor represents the combined influence of effects that are not explicitly modeled, including the impact of I-Term Relax. The factor is further applied to both the original analytical PI controller and the newly designed PI controller:

$$C_{\mathrm{PI,ana,new}}(\omega) = C_{\mathrm{PI,ana}}(\omega) \cdot C_{\mathrm{PI,com}}(\omega) \tag{2.25}$$

$$C_{\mathrm{PI,new,new}}(\omega) = C_{\mathrm{PI,new}}(\omega) \cdot C_{\mathrm{PI,com}}(\omega) \tag{2.26}$$

The resulting compensated controller transfer functions are then reinserted into the closed-loop system for further analysis and evaluation [**bf_compITerm**].

2.4.4 Step Response Analysis

To be added.

2.4.5 Theoretical Plant Models

The goal of this thesis is to analyze the control loops of the Betaflight flight controller based on measurements obtained from flight experiments. To validate the proposed measurement and analysis methods, theoretical plant models are introduced as reference systems.

By comparing the experimentally identified system behavior with the responses of these theoretical plants, the accuracy and reliability of the applied analysis techniques can be assessed. Furthermore, the models provide a clear framework for understanding the interaction between the different control loops implemented in Betaflight.

The following subsections describe the individual plant models corresponding to the main control layers of the flight controller: angular rate, angle, altitude hold and position hold.

2.4.5.1 Motor Dynamics

As the basis of the whole system, the transfer function of the motor dynamics needs to be determined.

The motor–propeller system is modeled using the well-established electromechanical model of a DC motor. This model can also be applied to brushless DC (BLDC) motors in an equivalent form. It consists of one electrical and one mechanical differential equation.

The electrical behavior of the motor is described by

$$u(t) = R\,i(t) + L\,\frac{di(t)}{dt} + k_e\,\omega(t), \tag{2.27}$$

where $u(t)$ is the applied motor voltage, $i(t)$ the motor current, R the winding resistance, L the winding inductance, k_e the back electromotive force constant, and $\omega(t)$ the angular velocity.

The mechanical dynamics are given by

$$J_\mathrm{m}\,\frac{d\omega(t)}{dt} = k_t\,i(t), \tag{2.28}$$

where J_m denotes the combined moment of inertia of the motor and the propeller, and k_t is the torque constant. For simplicity, viscous friction and the aerodynamic load torque are neglected, resulting in a linear and tractable model.

Applying the Laplace transform to Eqs. (2.27) and (2.28) and eliminating the current $i(s)$ leads to the transfer function from the applied voltage to the angular velocity:

$$G_\mathrm{motor}(s) = \frac{\Omega(s)}{U(s)} = \frac{k_t}{J_\mathrm{m}Ls^2 + J_\mathrm{m}Rs + k_tk_e}. \tag{2.29}$$

The torque constant and the back electromotive force constant have the same numerical value. Therefore, the transfer function can be simplified to

$$G_\mathrm{motor}(s) = \frac{N(s)}{U(s)} = \frac{k}{J_\mathrm{m}Ls^2 + J_\mathrm{m}Rs + k^2}, \tag{2.30}$$

This transfer function represents a second-order (PT2) system, where both the electrical and mechanical dynamics of the motor are taken into account. However, the electrical dynamics typically influence the system only at higher frequencies due to the small electrical time constant. Therefore, in many practical applications, the inductance can be neglected, and the motor can be approximated by a first-order (PT1) system. This simplified model forms the basis for the subsequent analysis and controller design.

2.4.5.2 Plant Angular Rate

The basis of the flight control system is the angular rate controller, which regulates the rotational speed of the drone around its axes. It represents the innermost control loop and directly influences the performance of the outer loops, such as the angle and position controllers. Therefore, the plant model of the angular rate controller serves as the fundamental reference for the analysis of the entire control system.

Based on the motor dynamics described in the previous section, the plant of the angular rate controller can be derived. The input to this plant is the control signal generated by the controller, corresponding to the voltage applied to the motors, while the output is the resulting angular rate of the drone.

The transfer function between the input voltage and the motor speed is given by the motor model introduced earlier. However, the relationship between the motor speed and the angular rate of the drone is additionally influenced by the drone geometry and the aerodynamic forces generated by the propellers. The thrust produced by each propeller can be approximated by

$$T = C_T \, \rho \, n^2 \, D^4 \tag{2.31}$$

where T is the thrust in Newtons (N), C_T is the dimensionless thrust coefficient depending on the propeller geometry and operating conditions, ρ is the air density in kg/m^3, n is the rotational speed in revolutions per second (1/s), and D is the propeller diameter in meters (m).

As already described, to roll or pitch the drone, the motors on one side of the drone are sped up while the motors on the opposite side are slowed down. This creates a torque that causes the drone to rotate around its axes. To calculate the torque generated by the motors, the thrust is multiplied by the distance from the center of mass to the motor, which is denoted as l. The resulting torque can be expressed as

$$M_{\text{roll}} = l \cdot \Delta T \tag{2.32}$$

where ΔT is the difference in thrust between the motors on opposite sides of the drone. This torque causes the drone to rotate and is directly related to the angular acceleration. The angular acceleration resulting from the applied torque can be obtained by rearranging Newton's second law for rotational motion:

$$\alpha_{\text{roll}} = \frac{M_{\text{roll}}}{J_{\text{roll}}}, \tag{2.33}$$

where M_{roll} is the applied torque and J_{roll} is the moment of inertia of the drone about the roll axis.

From this point, the angular acceleration only has to be integrated once to obtain the angular rate. Therefore, the transfer function from the control signal to the angular rate can be expressed as

$$P_{\text{Angular Rate}}(s) = \frac{\Omega_{\text{roll}}(s)}{U(s)} = \frac{\Omega(s)}{U(s)} = \frac{k_t}{J_{\text{m}} L s^2 + J_{\text{m}} R s + k_t k_e} \cdot \frac{l}{J_{\text{roll}}\, s} \tag{2.34}$$

where $\Omega_{\text{roll}}(s)$ is the angular rate of the drone about the roll axis and $U(s)$ is the control input corresponding to the applied motor voltage. The parameter k denotes the motor torque and back electromotive force (EMF) constant. L and R represent the inductance and resistance of the motor windings, respectively. J_{m} is the moment of inertia of the motor and propeller, and b is the viscous friction coefficient of the motor. The term J_{roll} denotes the moment of inertia of the drone about the roll axis. Finally, k_M is an aggregated gain that relates changes in motor speed to the generated torque acting on the drone, incorporating the propeller thrust coefficient, air density, propeller diameter, and the lever arm between the motor and the center of mass.

This transfer function results in a Bode plot with a slope of -20 dB/dec at low frequencies, which corresponds to the integrator behavior between torque and angular rate. At higher frequencies, the slope approaches -40 dB/dec with an additional phase lag tending towards -180°, primarily due to the inductive effects of the motor windings.

2.4.5.3 Plant Angle

The angle plant is considerably simpler than the angular rate plant. It is based on the fundamental kinematic relationship between the angular rate and the orientation of the multirotor. The angle corresponds to the time integral of the angular rate, which can be expressed as

$$\theta(t) = \int \omega(t)\, dt. \tag{2.35}$$

Assuming zero initial conditions and applying the Laplace transform, this relationship becomes

$$\Theta(s) = \frac{1}{s}\, \Omega(s). \tag{2.36}$$

This shows that the angle is obtained by integrating the angular rate, corresponding to a multiplication by $\frac{1}{s}$ in the Laplace domain. Consequently, from a theoretical point of view, the angle plant behaves as an ideal integrator with the transfer function

$$P_{\text{Angle}}(s) = \frac{\theta(s)}{\omega(s)} = \frac{1}{s}. \tag{2.37}$$

Through that calculations, we know that we can expect a bode plot of the angle plant with a slope of -20 dB/decade and a phase shift of -90 degrees, which are characteristic features of an ideal integrator.

2.4.5.4 Plant Position Hold

The position hold controller includes the angular rate and the angle as inner control loops. Therefore, the plant of the position hold corresponds to the transformation from the drone's attitude (angle) to its position in space. This transformation is determined by the kinematics of the multirotor and the influence of external forces such as gravity.

The position of the drone can be described by its coordinates in three-dimensional space, typically denoted as (x, y, z). The orientation of the drone influences how the thrust generated by the motors is directed and thus determines its motion in space.

If the drone is hovering at a constant altitude, the total thrust equals the weight of the drone, given by mg, where m is the mass of the drone and g is the gravitational acceleration. When the drone tilts by a small angle θ, a horizontal component of the thrust is generated, leading to a horizontal acceleration, as illustrated in Figure 2.15.

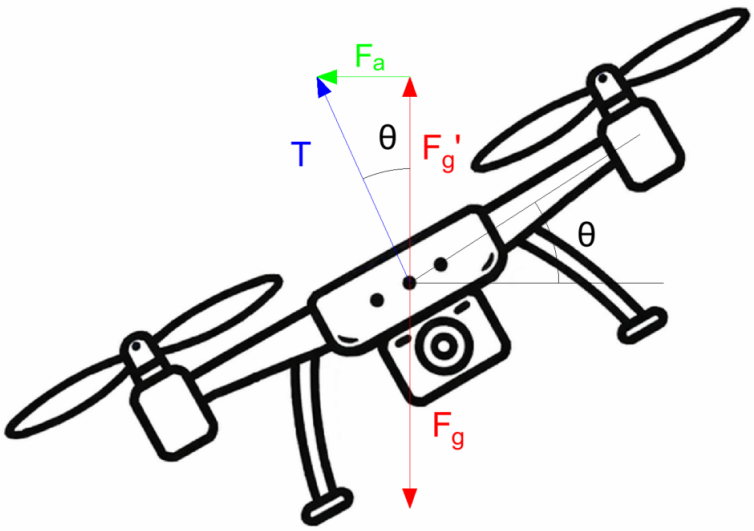

Figure 2.15: Illustration of a quadcopter tilted by an angle θ during position hold. The tilt redirects the thrust vector, generating a horizontal acceleration that results in translational motion.

The horizontal acceleration can be expressed as

$$a_{\text{hor}}(t) = g \cdot \sin(\theta(t)) \,. \tag{2.38}$$

Assuming that the drone operates at small angles during position hold, the small-angle approximation $\sin(\theta) \approx \theta$ can be applied. This results in the simplified expression

$$a_{\text{hor}}(t) \approx g \cdot \theta(t). \tag{2.39}$$

By integrating the horizontal acceleration twice with respect to time, the horizontal position of the drone is obtained:

$$x(t) = \int\int a_{\text{hor}}(t)\, dt^2 \approx \int\int g \cdot \theta(t)\, dt^2. \tag{2.40}$$

Applying the Laplace transform yields the transfer function from the tilt angle to the horizontal position:

$$P_{\text{Position}}(s) = \frac{X(s)}{\Theta(s)} = \frac{g}{s^2}. \tag{2.41}$$

This transfer function represents a double integrator. Consequently, the corresponding Bode plot exhibits a magnitude slope of $-40\,\text{dB/decade}$ and a phase shift of $-180°$, which are characteristic features of such a system.

2.4.5.5 Plant Altitude Hold

The altitude hold is the only control loop in this thesis that does not include the angle and angular rate control loops as inner loops. Instead, it directly controls the vertical position of the drone by adjusting the total thrust generated by the motors. Therefore, the plant of the altitude hold describes the transformation from the motor speed to the vertical position of the drone.

Similar to the other control loops, the altitude hold plant also includes the motor dynamics. Hence, one part of the plant is given by the motor transfer function introduced in the first section. The second part describes the transformation from motor speed to vertical motion.

The thrust produced by a propeller can be approximated by

$$T = C_T \rho n^2 D^4, \tag{2.42}$$

where T is the thrust, C_T is the thrust coefficient, ρ is the air density, n is the rotational speed, and D is the propeller diameter.

The vertical acceleration of the drone is obtained from the sum of forces in vertical direction:

$$a_{\text{ver}}(t) = \frac{T(t)}{m} - g. \tag{2.43}$$

For hover, the thrust balances the weight of the drone, i.e. $T_0 = mg$. Considering small deviations around this operating point, the system can be linearized, which yields

$$\Delta a_{\text{ver}}(t) = \frac{\Delta T(t)}{m}. \tag{2.44}$$

Since the thrust depends quadratically on the motor speed, linearization around the operating point n_0, which corresponds to the motor rotational speed in hover, gives

$$\Delta T(t) \approx 2C_T \rho D^4 n_0 \, \Delta n(t). \tag{2.45}$$

By integrating the vertical acceleration twice, the vertical position is obtained. Applying the Laplace transform results in the transfer function

$$P_{\text{Altitude}}(s) = \frac{\Delta S(s)}{\Delta N(s)} = \frac{2C_T \rho D^4 n_0}{m s^2} \, P_{\text{Motor}}(s). \tag{2.46}$$

Thus, the altitude plant consists of the motor dynamics and a double integrator describing the vertical motion. The Bode plot of this plant exhibits a slope of $-40\,\text{dB/decade}$ at low frequencies due to the double integrator, while the motor dynamics introduce additional phase lag and magnitude reduction at higher frequencies.

3 Methodology

In this chapter, the practical application of the theoretical concepts introduced in the previous chapter is described. The focus is on how these principles are implemented in the context of tuning the control loops of a Betaflight-based flight controller. The methodology is structured around the different control loops, starting with the angle controller, followed by the altitude hold controller and the position controller. For each control loop, the specific implementation details and tuning procedures are presented.described.

3.1 Angle Controller Tuning

The Betaflight software provides three main flight modes: Angle, Horizon, and Acro.

In Angle mode, the drone's tilt angle is directly linked to the stick position. The maximum tilt is limited to a predefined value, preventing flips and making this mode particularly suitable for beginners.

In contrast, Acro mode provides full manual control. In this mode, stick inputs define an angular rate rather than a target angle, meaning that the drone continues rotating as long as the stick remains deflected. This allows aggressive maneuvers such as flips and rolls but also requires greater pilot experience.

During this thesis project, an offline tuner for the angular rate controller was further developed to systematically optimize the rate control loop and improve Acro mode performance. The tuner was subsequently extended to also support Angle mode, enabling improved stability and responsiveness while maintaining intuitive handling for beginner pilots.

3.1.1 Betaflight Firmware

This chapter discusses how the Betaflight firmware is implemented and what modifications were made to the original codebase in order to integrate the new tuning methods.

3.1.1.1 Angle Mode Control Loop

In Angle mode, the angular rate controller is extended by an outer angle control loop, resulting in a cascaded control structure.

The stick input initially generates a rate setpoint. Unlike in Acro mode, this rate setpoint is then converted into a target angle. This target angle serves as the reference input for the inner angular rate controller.

The cascaded architecture separates attitude control from angular rate stabilization. While the outer loop ensures convergence toward the desired orientation, the inner loop provides fast dynamic stabilization by regulating the angular velocity. As a result, the system achieves

improved disturbance rejection and more intuitive handling characteristics, since the pilot directly commands an angle rather than a rotational rate.

The Angle mode controller therefore consists of two cascaded loops:

- The outer angle loop, which computes the angle error.
- The inner angular rate loop, which computes the rate error.

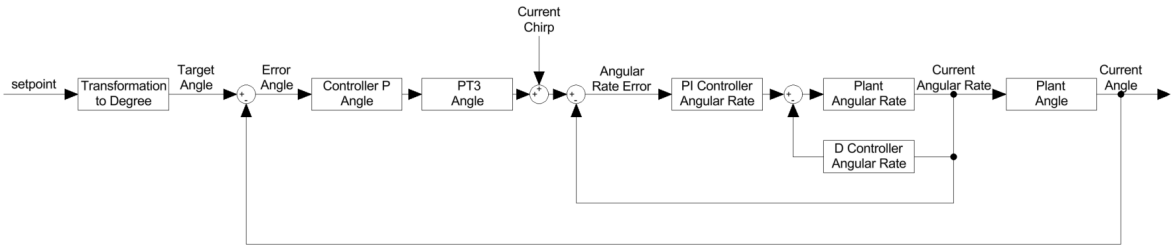

Figure 3.1: Cascaded control structure of the Betaflight Angle mode controller.

The angle control loop consists of a proportional controller with an additional feedforward term. Within the offline tuning framework, however, only the proportional controller is optimized. The feedforward path is included in the control structure for completeness but was deliberately disabled during the tuning process.

3.1.2 Derivation of the Target Angle

The target angle is obtained by scaling the commanded angular rate with respect to the maximum achievable rate and the configured maximum tilt angle:

$$\theta_{\text{target}} = \theta_{\text{limit}} \cdot \frac{\omega_{\text{set}}}{\omega_{\text{max}}} \tag{3.1}$$

where

- θ_{limit} is the maximum allowed tilt angle (e.g., 60°),
- ω_{set} is the commanded angular rate derived from the stick input,
- ω_{max} is the maximum achievable angular rate determined by the rate configuration.

This relation ensures a proportional mapping between stick deflection and the resulting target angle.

For example,

$$\omega_{\text{set}} = \omega_{\text{max}} \Rightarrow \theta_{\text{target}} = \theta_{\text{limit}} \tag{3.2}$$

$$\omega_{\text{set}} = 0 \Rightarrow \theta_{\text{target}} = 0 \tag{3.3}$$

Thus, the stick position scales the target angle continuously within the range

$$-\theta_{\text{limit}} \leq \theta_{\text{target}} \leq +\theta_{\text{limit}}. \tag{3.4}$$

3.1.2.1 Firmware Modifications

As already mentioned in the gyro control tuning section, a chirp signal is injected into the control loop to excite the system over a wide frequency range.

If the chirp signal were added in the same way as for the gyro control tuning, it would not be possible to directly calculate the transfer function of the complete system from the target angle to the measured angle. In such a configuration, the chirp signal would be added to the current setpoint and therefore inside the closed control loop. Consequently, the system would have two inputs: the stick input and the injected chirp signal. Additionally, the noise on the signal could not be filtered as described in the theory chapter.

This situation is illustrated in Figure **??**, which shows the control loop structure currently implemented in the Betaflight firmware.

To overcome this issue, a small modification to the firmware allows the chirp signal to be added directly to the setpoint before it is converted into the target angle. Due to the linear transformation from the rate setpoint to the target angle, the parameters of the chirp signal do not need to be adjusted.

The modified control structure used for the offline tuning framework is illustrated in Figure 3.2. In this configuration, the chirp signal is injected before the transformation from the rate setpoint to the target angle, allowing the transfer function from the target angle to the measured angle to be estimated directly.

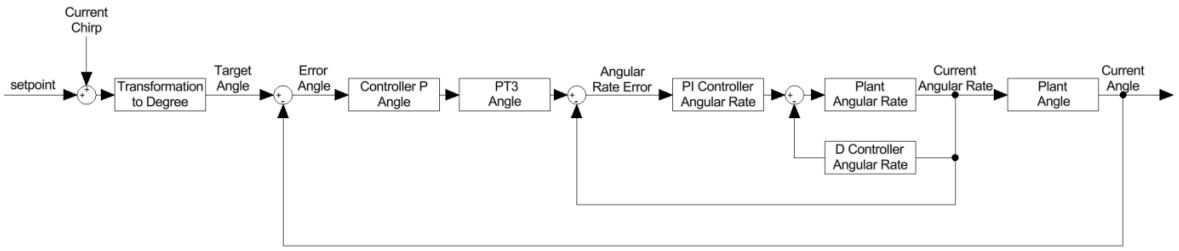

Figure 3.2: Cascaded control structure of the Betaflight Angle mode controller.

Some older drones are only capable of flying in Acro mode and therefore do not support Angle mode. In this case, the chirp signal can still be added to the setpoint. Since the angle control loop and the transformation from the rate setpoint to the target angle are disabled in Acro mode, the resulting control loop structure is identical to the one used for the gyro control tuning.

This approach allows the same chirp signal to be used for both modes. As a result, the implementation is simplified and the tuning process becomes more straightforward, since the same excitation signal can be applied without considering structural differences in the control loop.

3.1.3 Control Loop Calculation

Based on the modified control structure, the dynamic behaviour of the Angle mode control loop can be analysed using the recorded input and output signals. Due to the chirp signal being injected before the transformation to the target angle, the system can be treated as a single-input single-output (SISO) system from the target angle to the measured angle. This allows the same signal processing steps as described in the theory chapter to be applied, including noise filtering and windowing.

3.1.3.1 Estimation of the Closed-Loop Components

Using the Fourier transforms of the target angle $\Theta_{\text{target}}(\omega)$ and the measured angle $\Theta(\omega)$, the closed-loop transfer function is obtained as

$$G_{cl}(\omega) = \frac{\Theta(\omega)}{\Theta_{\text{target}}(\omega)}. \tag{3.5}$$

This transfer function describes the complete behaviour of the cascaded control structure, including both the outer angle controller and the inner angular rate controller, as well as the plant dynamics.

As described in the theory chapter, it is also necessary to estimate the transfer function from the target angle to the angular rate $\Omega(\omega)$:

$$G_{wy}(\omega) = \frac{\Omega(\omega)}{\Theta_{\text{target}}(\omega)}. \tag{3.6}$$

With the two transfer functions $G_{cl}(\omega)$ and $G_{wy}(\omega)$, the plant of the angle control loop can be calculated as follows:

$$G_{\text{plant}}(\omega) = \frac{G_{cl}(\omega)}{G_{wy}(\omega)}. \tag{3.7}$$

This transfer function forms the foundation for the subsequent tuning of the angle control loop. With the measured plant dynamics, all the necessary information is available to optimize the controller parameters and improve the performance of the Angle mode.

3.1.3.2 Implementation of the Tuning Method

The tuning method for the angle control loop is based on the estimated plant dynamics. By analysing the frequency response of the plant, the controller parameters can be adjusted to achieve the desired performance characteristics, such as stability margins and bandwidth.

As all components of the control loop are now known—including the plant and the inner angular rate control loop—the new closed-loop transfer function for a set of updated controller parameters can be calculated as

$$G_{cl,\text{new}}(\omega) = \frac{C_{\text{new}}(\omega)\, G_{\text{Gyro}}(\omega)\, G_{\text{plant}}(\omega)}{1 + C_{\text{new}}(\omega)\, G_{\text{Gyro}}(\omega)\, G_{\text{plant}}(\omega)}. \tag{3.8}$$

Furthermore, the sensitivity function, which characterizes the disturbance rejection and robustness of the control system, can be expressed as

$$S_{\text{new}}(\omega) = \frac{1}{1 + C_{\text{new}}(\omega)\, G_{\text{Gyro}}(\omega)\, G_{\text{plant}}(\omega)}. \tag{3.9}$$

These expressions allow the computation of all relevant performance metrics, such as gain and phase margins, bandwidth, and disturbance attenuation. Based on these metrics, plots can be generated and the suitability of the new controller parameters can be systematically evaluated.

3.1.4 Drone Tuning

Declaration of Generative AI Use

The following generative AI systems were used during the preparation of this thesis. Their use was limited to the functions described below. All generated content was critically reviewed and verified by the authors.